

Appunti sul progetto di Pneumonia Detection

Contents

1	ResNet	2
1.1	Bottleneck	2
1.1.1	Batch Normalization	3
1.2	Struttura di ResNet-50	3
1.2.1	Global Average Pooling	3

1 ResNet

La ResNet (Residual Network) introduce le *residual connections* per facilitare l'addestramento di reti molto profonde. L'idea è che gli strati convoluzionali non debbano necessariamente apprendere direttamente la funzione desiderata $H(x)$, ma possano apprendere una funzione residua $F(x)$ rispetto all'input:

$$F(x) = H(x) - x$$

Il risultato del blocco viene quindi ottenuto come:

$$H(x) = F(x) + x$$

L'input x viene quindi mantenuto attraverso una *shortcut connection* e sommato all'output della trasformazione residua. Questa struttura facilita l'ottimizzazione delle reti profonde, poiché, nel caso in cui la trasformazione ottimale fosse vicina all'identità, il blocco deve imparare una correzione rispetto all'input invece dell'intera trasformazione. Il paper originale introduce questa formulazione come principio fondamentale della residual learning.

1.1 Bottleneck

ResNet-50 utilizza un particolare tipo di residual block chiamato *bottleneck block*. Il blocco è costituito da tre convoluzioni:

$$1 \times 1 \rightarrow 3 \times 3 \rightarrow 1 \times 1$$

La prima convoluzione 1×1 riduce il numero di canali della rappresentazione. La convoluzione 3×3 , che è la principale convoluzione spaziale del blocco, opera quindi su un numero ridotto di canali, diminuendo il costo computazionale. Infine, la seconda convoluzione 1×1 aumenta nuovamente il numero di canali, riportandolo alla dimensione necessaria per effettuare la somma con la shortcut.

Ad esempio, un blocco può trasformare una rappresentazione:

$$256 \rightarrow 64 \rightarrow 64 \rightarrow 256$$

dove il passaggio a 64 canali costituisce il *bottleneck*.

La struttura completa del blocco può essere rappresentata come:

$$x \rightarrow 1 \times 1 \rightarrow BN \rightarrow ReLU \rightarrow 3 \times 3 \rightarrow BN \rightarrow ReLU \rightarrow 1 \times 1 \rightarrow BN$$

e, in parallelo, l'input viene trasmesso attraverso la shortcut. I due rami vengono poi sommati e viene applicata una ReLU finale:

$$y = ReLU(F(x) + x)$$

Quando le dimensioni dell'input e dell'output non coincidono, la shortcut viene trasformata tramite una convoluzione 1×1 per adattarne numero di canali e, se necessario, risoluzione spaziale.

Il bottleneck è stato introdotto nel paper originale proprio per rendere più efficiente la costruzione di reti molto profonde: le convoluzioni 1×1 riducono e poi aumentano le dimensioni, lasciando alla convoluzione 3×3 una rappresentazione più piccola su cui operare.

1.1.1 Batch Normalization

La *Batch Normalization* (BN) normalizza le attivazioni prodotte da una convoluzione utilizzando la media e la varianza calcolate sul mini-batch. Successivamente, le attivazioni normalizzate vengono trasformate tramite due parametri apprendibili, che permettono alla rete di adattarne nuovamente scala e traslazione.

In ResNet, la Batch Normalization viene applicata dopo ogni convoluzione e prima della funzione di attivazione ReLU. La sua funzione è rendere più stabile l'addestramento della rete e facilitare l'ottimizzazione dei numerosi strati convoluzionali.

1.2 Struttura di ResNet-50

ResNet-50 è costruita come una sequenza di quattro stage principali, ciascuno composto da più Bottleneck block. La configurazione di ResNet-50 utilizza rispettivamente:

$$[3, 4, 6, 3]$$

Bottleneck nei quattro stage `conv2_x`, `conv3_x`, `conv4_x` e `conv5_x`.

La struttura può essere riassunta come:

$$\text{Input} \rightarrow 7 \times 7 \text{ Conv} \rightarrow \text{MaxPool} \rightarrow \text{conv2_x} \rightarrow \text{conv3_x} \rightarrow \text{conv4_x} \rightarrow \text{conv5_x}$$

Per un input RGB di dimensione 224×224 , le principali dimensioni delle feature map sono:

$$224 \times 224 \rightarrow 112 \times 112 \rightarrow 56 \times 56 \rightarrow 28 \times 28 \rightarrow 14 \times 14 \rightarrow 7 \times 7$$

mentre il numero di canali aumenta progressivamente:

$$64 \rightarrow 256 \rightarrow 512 \rightarrow 1024 \rightarrow 2048.$$

Il primo layer utilizza una convoluzione 7×7 con stride 2, seguita da un max pooling con stride 2. I successivi cambiamenti di risoluzione avvengono all'inizio degli stage `conv3_x`, `conv4_x` e `conv5_x`, dove il primo Bottleneck utilizza stride 2. Nel codice, quindi, il primo Bottleneck di ogni nuovo stage è anche quello che adatta la dimensione della shortcut quando necessario.

1.2.1 Global Average Pooling

Al termine della rete, la ResNet originale utilizza un *Global Average Pooling* (GAP) per trasformare la feature map finale in un vettore compatto.

Nel caso di ResNet-50, la feature map finale ha dimensione:

$$7 \times 7 \times 2048.$$

Il Global Average Pooling calcola, per ogni canale, la media dei valori presenti su tutte le posizioni spaziali:

$$y_c = \frac{1}{HW} \sum_{i=1}^H \sum_{j=1}^W x_{c,i,j}.$$

La dimensione spaziale viene quindi ridotta da 7×7 a 1×1 , ottenendo:

$$7 \times 7 \times 2048 \rightarrow 1 \times 1 \times 2048.$$

Successivamente il tensore viene appiattito e passato al layer fully connected che produce le predizioni finali.

Il vantaggio del Global Average Pooling è che permette di riassumere l'intera mappa spaziale di ciascun canale in un singolo valore, evitando l'utilizzo di una grande quantità di parametri tipica di un fully connected applicato direttamente alla feature map. Il GAP non verrà usato nel caso in cui ResNet debba funzionare come backbone.